

Network Operations and Predictive Intelligence System

Project Documentation

Implementation covering Core Python PySpark Data Engineering FastAPI React and Machine Learning

Done by: Yuvasree R

Contents

- 1 Project Introduction
- 2 Project Objectives
- 3 Dataset and Data Understanding
- 4 Overall System Architecture
- 5 Phase 1 Core Python
- 6 Phase 2 PySpark Distributed Processing
- 7 Phase 3 Data Engineering
- 8 Phase 4 FastAPI Service Layer
- 9 Phase 5 React NOC Dashboard
- 10 Phase 6 Machine Learning

1 Project Introduction

The Network Operations and Predictive Intelligence System was implemented as an end-to-end analytics platform for exploring geographically indexed telecommunications activity in Milan. The work converts daily activity extracts into curated data, reusable analytics, operational alerts, machine-learning features, risk scores, API services, and a browser-based Network Operations Center dashboard. The project therefore demonstrates how a data product can progress from local exploratory processing to distributed transformation, orchestrated data engineering, service delivery, visualization, and operational machine learning while retaining a consistent data contract across every layer.

The platform uses the Telecom Italia Mobile Phone Activity dataset together with the Milan grid reference. The activity data records proportional measures for incoming and outgoing SMS activity, incoming and outgoing call activity, and internet activity. These measures are associated with a geographic grid cell, a timestamp, and a country-code category at the raw grain. The operational analytics path intentionally removes the country-code dimension by summing all categories for each grid and hourly timestamp. This design produces one authoritative grid-hour record and prevents downstream key performance indicators, features, alerts, and model scores from being inflated by duplicate country-code rows.

The completed implementation is organized into six phases. Core Python establishes the data semantics, data-quality rules, aggregation logic, and rule-based alerting. PySpark reproduces the same business rules using distributed DataFrames and introduces schema enforcement, execution-plan analysis, partitioning, caching, and geographic enrichment. Data Engineering turns the individual transformations into landing, raw, processed, analytics, and warehouse workflows coordinated by Airflow. FastAPI exposes the curated information through stable request and response contracts. React consumes those contracts in an operational dashboard. Machine Learning adds past-only feature engineering, next-hour high-activity risk prediction, historical anomaly detection, and serving of model results through the same API and user interface.

The system is an activity-intelligence platform rather than a radio-performance or capacity-monitoring system. The available data does not contain throughput, latency, packet loss, utilization, radio quality, or capacity information. Consequently, high activity is treated as an operational attention signal and not as proof of congestion or a network fault. This wording is preserved in the alert logic, model framing, API responses, and dashboard presentation.

2 Project Objectives

The primary objective was to build a coherent analytics system in which every phase consumes trusted products from the preceding phase. The implementation was designed to preserve raw evidence, standardize field names and data types, enforce the grid-hour grain, retain traceability to source files, and expose the most recent dataset timestamp through a shared AS OF convention. These foundations allow analytical results to be reproduced and compared across pandas, Spark, SQL, APIs, and the frontend.

The system was also intended to provide useful operational views without overstating what the dataset can prove. Network-wide summaries identify total composite activity, active grids, peak hour, and the highest-activity grid. Grid drill-down exposes the individual activity measures through time. Hotspot and alert views identify cells that deserve attention and place them on the true Milan grid geometry. The predictive view estimates whether the following hour is likely to show high activity and supplements that forecast with an independent historical anomaly assessment and the earlier rule-based alerts.

Objective	Implemented capability
Reliable data foundation	Canonical schema, key validation, null handling, negative-value rejection, and unchanged raw-file preservation
Scalable processing	Multi-file PySpark ingestion, grid-hour aggregation, optimized joins, Parquet outputs, and reusable ETL functions
Production-style pipeline	Landing-to-raw routing, Airflow orchestration, warehouse loading, freshness checks, and machine-readable status
Operational access	FastAPI endpoints for summaries, grids, hotspots, alerts, features, predictions, pipeline status, locations, and neighbours
Decision support	React NOC dashboard, rule-based alerts, XGBoost risk scoring, and historical anomaly comparison

3 Dataset and Data Understanding

3.1 Source Data

The project uses the Milan subset of the Telecom Italia Mobile Phone Activity data and the corresponding milano-grid GeoJSON reference. Only Milan activity files matching the sms-call-internet-mi date-based naming convention are intended for the pipeline. Province-level directional call files, census information, social or weather data, the Trentino activity files, and the Trentino grid are outside the implemented scope because their inclusion would change the semantics or corrupt the geographic join.

Each supplied daily activity file contains 24 hourly intervals. At the source grain, several country-code records may exist for the same grid and timestamp. A raw row therefore does not represent the final operational unit. The canonical analytical unit is one geographic grid cell during one hourly interval. Country-code detail is preserved in the raw and cleaned layers but is deliberately excluded from the operational aggregation, warehouse fact table, APIs, features, alerts, and dashboard.

3.2 Core Schema

Canonical field	Meaning and treatment
timestamp	Hourly observation time parsed from the source datetime field
grid_id	Milan grid-cell identifier normalized from CellID and constrained to the 1 to 10000 range
country_code	Raw categorical dimension retained before grid-hour aggregation
sms_in and sms_out	Incoming and outgoing SMS activity measures
call_in and call_out	Incoming and outgoing call activity measures
internet_activity	Internet activity measure; frequently the largest component of the composite indicator
total_sms	sms_in plus sms_out
total_calls	call_in plus call_out

Canonical field	Meaning and treatment
total_activity	total_sms plus total_calls plus internet_activity; a project-defined composite indicator

3.3 Data Quality and Time Conventions

Missing grid identifiers and invalid timestamps are treated as key failures because the affected rows cannot be assigned to a reliable geographic or temporal grain. Negative activity measures are also rejected as impossible values for the supplied data. Blank activity cells are handled differently: they are profiled first and then converted to zero in the curated layer so an otherwise valid observation is not discarded. The number of replacements is retained as a pipeline-quality metric.

The AS OF value is defined as the maximum timestamp present in the curated analytics data, not the current computer time. This distinction is necessary because the source data is historical while the application is presented as an operational dashboard. The Airflow status record, warehouse services, FastAPI responses, and React pages use this dataset-derived timestamp to keep every layer synchronized.

3.4 Geographic Reference

The Milan reference is a GeoJSON FeatureCollection containing 10,000 polygon cells arranged as a 100 by 100 grid in WGS 84 coordinates. The correct join key is properties.cellld. The GeoJSON top-level feature id is zero-based and must not be used; using it would create an off-by-one geographic error while still appearing to achieve almost complete join coverage. The Spark enrichment and React map both normalize properties.cellld to grid_id, and geographic spot checks are used to confirm that selected centroids fall within the Milan area.

4 Overall System Architecture

The architecture follows a layered flow in which files move forward while the original evidence remains unchanged. Daily CSV files enter a landing area and are validated before being copied to raw or rejected storage. PySpark canonicalizes the accepted data, collapses it to grid-hour grain, calculates analytics, and enriches activity records with the Milan grid reference. Curated Parquet datasets are then loaded into a MySQL dimensional warehouse. Feature generation and batch intelligence operate from this trusted history. FastAPI reads the warehouse, stored features, model artifacts, anomaly scores, and pipeline status. React consumes the API and the static GeoJSON reference to provide network, grid, map, alert, and predictive views.

Layer	Responsibility	Principal implementation
Landing and raw	File detection, schema checks, quality checks, routing, and immutable source preservation	de2_ingestion.py and DE2 Airflow DAG
Distributed processing	Canonicalization, cleaning, aggregation, geographic enrichment, and Parquet publication	spark modules and telecom_pipeline.py
Orchestration	Task dependencies, run status, failure propagation, freshness, and quality checks	de3_processing_dag.py and de7_end_to_end.py
Warehouse	Time and grid dimensions plus grid-hour activity	de6_warehouse.py and MySQL

Layer	Responsibility	Principal implementation
	facts	
Intelligence	Past-only features, next-hour risk classification, anomaly scoring, and comparisons	ML2, ML3, and ML4 modules
Services	Validated operational contracts and database-backed queries	FastAPI routers, Pydantic models, and services
Presentation	KPI cards, time-series exploration, geographic prioritization, and prediction display	React, Recharts, and Leaflet

5 Phase 1 Core Python

Phase 1 established the project's data contract with a dataset small enough to inspect directly. Pandas was used to profile one daily file, implement a reusable processing class, and create explainable rules for unusual activity. This phase defined the canonical names, rejection logic, null policy, derived activity measures, and grid-hour grain that were preserved throughout later phases.

5.1 NP1 Telecom Activity Dataset Profiling

The first lab examined the daily activity extract before reusable processing was introduced. The schema, data types, timestamp cadence, row count, grid coverage, country-code categories, missing values, negative values, duplicate patterns, and activity distributions were profiled. The hourly cadence was verified from the file rather than assumed from the historic source publication, whose original release used shorter time slots. This ensured that later expectations such as 24 intervals per daily file were based on the supplied data.

The profiling work clarified the difference between raw and operational grain. Multiple rows may share a timestamp and grid because they belong to different country-code categories. The operational dataset therefore requires a group-by on timestamp and grid_id with sums of the five activity measures. During profiling, total_sms, total_calls, and total_activity were derived to evaluate component contributions and to identify the extent to which internet activity dominates the composite value. The individual measures were retained so the dashboard and API would not reduce the source to one ambiguous number.

The analysis also established the project's terminology. The values are proportional activity measures rather than literal message counts, call counts, or data volume. A grid is a geographic cell rather than a tower, subscriber, or customer. These distinctions are important because the application later presents hotspots and model outputs to operational users.

```
print("Shape:", df.shape)
print("\nRaw Columns:", df.columns.tolist())
print("\nData Types:", df.dtypes)
```

Python

Shape: (1891928, 8)

Raw Columns: ['datetime', 'CellID', 'countrycode', 'smsin', 'smsout', 'callin', 'callout', 'internet']

Data Types: datetime str
CellID int64
countrycode int64
smsin float64
smsout float64
callin float64
callout float64
internet float64
dtype: object

```

=====
NP1 PROFILE SUMMARY
=====
Rows: 1891928
Columns: 14
Unique grids: 10000
Distinct timestamps: 24
Time range: 2013-11-01 00:00:00 to 2013-11-01 23:00:00
Country codes: 246
Exact duplicates: 0
Missing grid_id: 0
Missing timestamp: 0

Null counts:
timestamp          0
grid_id            0
country_code       0
sms_in             1086153
sms_out            1422446
call_in            1407781
call_out           1037413
internet_activity  1087074
date               0
hour               0
day_of_week        0
dtype: int64

```

5.2 NP2 UsageProcessor Class

A reusable UsageProcessor class was implemented in np2_UsageProcessor.py to convert the exploratory logic into an ordered, testable processing component. The class accepts either a file path or an existing pandas DataFrame. Its methods load data, rename raw columns, validate required fields, parse timestamps, convert activity measures to numeric values, reject missing keys and negative activities, replace remaining activity nulls with zero, derive calendar fields, aggregate country-code rows, calculate activity features, compute summaries, and export the results.

State is retained between methods through clean_df, grid_hour_df, daily_summary, and grid_summary. The class also records rejected_rows and nulls_handled so the data-quality effects of processing are visible rather than silently absorbed. The aggregation method explicitly omits country_code and produces one record per timestamp and grid_id. It then creates total_sms, total_calls, total_activity, date, hour, and day_of_week fields on the aggregated data.

The KPI method produces a daily summary and a grid summary, identifies the top ten grids by activity, finds the peak timestamp, and calculates the share of internet activity in the composite indicator. The export method writes daily_summary.csv, grid_summary.csv, and grid_hour_summary.csv. A companion test module constructs controlled data and exercises loading, cleaning, time derivation, grid-hour aggregation, activity derivation, and KPI computation, which makes the class suitable for reuse in later alert-generation work.

5.3 NP3 Rule Based Network Activity Alerts

The rule-based alert generator reuses UsageProcessor and evaluates each grid-hour record against an explainable within-day baseline. For every grid and date, a leave-one-out median is calculated: the baseline for the current hour is the median of the other hours in the same grid-day. Leaving the current observation out prevents an extreme value from directly raising its own reference value.

Low-volume grid-days can generate unstable ratios, so the lowest 15 percent of daily grid activity is excluded using a data-derived activity floor. The current activity is then compared with both the baseline and the previous hour. HIGH_ACTIVITY is raised when the current value reaches at least twice the baseline. ACTIVITY_SPIKE is raised when it reaches at least twice the previous hour. ACTIVITY_DROP is raised when it falls to one half or less of the baseline. Each event includes the grid, timestamp, current value, baseline, alert type, severity, and a human-readable reason.

The single-day implementation writes CSV and JSON alert collections and summarizes the alert distribution and alert rate. A multi-file database variant processes all accepted raw files, inserts alerts into fact_network_alerts by resolving time and grid dimension keys, marks high activity and spikes as HIGH severity, and marks drops as MEDIUM severity. The baseline remains intentionally within-day at this stage; the same-grid and same-hour historical baseline becomes possible only after multiple days have accumulated and is implemented in ML4.

	grid_id	timestamp	alert_type	current_activity	baseline_activity	reason
1	1	2013-11-01 03:00:00	ACTIVITY_DROP	35.0978	83.7437	Current activity (35) is only 0.42x baseline.
2	1	2013-11-01 04:00:00	ACTIVITY_DROP	32.2741	83.7437	Current activity (32) is only 0.39x baseline.
3	1	2013-11-01 05:00:00	ACTIVITY_DROP	35.416	83.7437	Current activity (35) is only 0.42x baseline.
4	1	2013-11-01 06:00:00	ACTIVITY_DROP	36.1216	83.7437	Current activity (36) is only 0.43x baseline.
5	2	2013-11-01 03:00:00	ACTIVITY_DROP	35.1605	84.2422	Current activity (35) is only 0.42x baseline.
6	2	2013-11-01 04:00:00	ACTIVITY_DROP	32.3617	84.2422	Current activity (32) is only 0.38x baseline.
7	2	2013-11-01 05:00:00	ACTIVITY_DROP	35.501	84.2422	Current activity (36) is only 0.42x baseline.
8	2	2013-11-01 06:00:00	ACTIVITY_DROP	36.225100000000005	84.2422	Current activity (36) is only 0.43x baseline.
9	3	2013-11-01 02:00:00	ACTIVITY_DROP	42.3475	84.7727	Current activity (42) is only 0.50x baseline.
10	3	2013-11-01 03:00:00	ACTIVITY_DROP	35.2271	84.7727	Current activity (35) is only 0.42x baseline.
11	3	2013-11-01 04:00:00	ACTIVITY_DROP	32.4546	84.7727	Current activity (32) is only 0.38x baseline.
12	3	2013-11-01 05:00:00	ACTIVITY_DROP	35.591499999999996	84.7727	Current activity (36) is only 0.42x baseline.
13	3	2013-11-01 06:00:00	ACTIVITY_DROP	36.3351	84.7727	Current activity (36) is only 0.43x baseline.
14	4	2013-11-01 03:00:00	ACTIVITY_DROP	34.9165	82.2863	Current activity (35) is only 0.42x baseline.
15	4	2013-11-01 04:00:00	ACTIVITY_DROP	35.0000	82.2863	Current activity (36) is only 0.38x baseline.

6 Phase 2 PySpark Distributed Processing

Phase 2 transferred the validated business rules from pandas to Spark. The objective was not to change the meaning of the data but to execute the same transformations across multiple daily files using distributed DataFrames, explicit schemas, lazy evaluation, partition-aware storage, and optimized joins. Individual learning scripts were complemented by a reusable command-line ETL module.

6.1 SP1 Distributed Ingestion

A SparkSession was created and daily Milan files were loaded through a wildcard pattern into a single DataFrame. A manual StructType defined the timestamp source as text, CellID as an integer, countrycode as text, and the five activity measures as doubles. This avoids repeated inference passes and provides a stable contract when new daily files arrive. The input_file_name function was added to every row so file-level lineage remains available after the files are combined.

The ingestion script reports the files discovered by the operating system and the distinct sources reported by Spark. It also checks the total row count, unique grids, country-code categories, distinct timestamps, expected number of hourly intervals, valid grid range, populated lineage field, and file-level record counts. The original partition count is inspected to connect physical file layout with Spark parallelism. The distributed raw result is persisted as Parquet for the next stage without collapsing the country-code grain.

6.2 SP2 Cleaning and Standardization

The raw Parquet dataset was renamed to the canonical schema and explicitly cast to timestamps, integers, and doubles. Activity nulls were profiled before modification. Rows with missing grid_id, missing timestamp, or any negative activity measure were separated as rejected records. Valid rows were retained, and remaining null activity cells were replaced with zero only after the rejected population had been counted.

Composite measures and calendar attributes were generated with Spark column expressions. The process reconciles the record counts before and after cleaning, reports the number of activity null cells handled, verifies the expected timestamp cadence, confirms the required measures remain present, and writes the clean DataFrame to Parquet. This reproduces the pandas rules while making the transformations declarative and distributable.

6.3 SP3 Network Activity Aggregations

Country-code records were collapsed by grouping on grid_id and timestamp and summing SMS, call, and internet measures. total_sms, total_calls, total_activity, internet_share, and date were derived after aggregation. A strict assertion verifies that no duplicate grid_id and timestamp combination remains, and country_code is asserted to be absent from the result. The reduced row count and an upper bound of daily files multiplied by 24 hours and 10,000 grids provide additional grain checks.

One source grid-hour was reproduced manually from its country-code records and compared field by field with the Spark aggregation. Daily grid summaries, overall top-activity grids, daily ranked hotspots using a window function, and the network peak activity hour were then calculated. The authoritative hourly result was saved as hourly_grid_parquet for enrichment and publication.

6.4 SP4 Geospatial Enrichment

The Milan GeoJSON was parsed and flattened into a 10,000-row Spark lookup containing `grid_id` and serialized geometry. The implementation explicitly maps `properties.cellId` to `grid_id` and rejects missing or duplicate identifiers. The lookup size is checked against the expected 10,000 cells before any join is performed.

A conventional join plan and a broadcast join plan are both inspected. Because the grid reference is small relative to the activity history, the lookup is broadcast to executors and left-joined to the hourly activity DataFrame. Row counts and distinct grid counts are checked before and after enrichment, unmatched cells are identified, and 100 percent geographic coverage is required. Centroids are calculated for sanity checks, including a known Milan cell and the expected proximity of adjacent grid cells. The enriched activity and grid lookup datasets are then stored as Parquet.

6.5 SP5 Performance and Execution Behaviour

Spark performance was investigated through physical plans and measured actions rather than by applying optimizations blindly. The hotspot aggregation plan was recorded, and cache behavior was tested on a pruned DataFrame containing only `grid_id`, `date`, and `total_activity`. `MEMORY_AND_DISK` persistence was selected after recognizing that caching the complete clean dataset with `MEMORY_ONLY` could exhaust the Java heap. The first cached action materializes the data, while the second demonstrates reuse against comparable uncached actions.

Partition behavior was explored by repartitioning on date with a modest partition count and by deliberately creating 200 partitions to demonstrate task-scheduling overhead on a small local workload. Full-column and column-pruned aggregations were compared, and standard versus broadcast geographic joins were examined through formatted plans and row-count equivalence. The tests reinforced that optimization choices should follow data size, reuse frequency, join cardinality, and execution evidence.

6.6 SP6 Processed and Analytics Data Publication

Curated and analytical results were published primarily as Parquet. The clean dataset, hourly grid summary, geospatial activity, and grid lookup were read back and checked for stable counts and schemas. The grid-hour duplicate assertion was repeated, and the non-geographic hourly summary was checked to ensure geometry had not been unnecessarily embedded in the compact analytical table.

A date-partitioned clean dataset was written to demonstrate partition pruning for date-based processing. A small dashboard summary was coalesced and exported as a single CSV containing daily SMS, call, internet, total activity, and active-grid metrics. File sizes were measured to illustrate why Parquet is better suited to analytical storage while CSV remains useful for compact interchange and inspection.

6.7 SP7 Reusable Spark ETL Job

The separate Spark activities were consolidated into `telecom_pipeline.py`. The module exposes functions for logging, Spark-session creation, raw reading, cleaning, aggregation, geographic enrichment, and output writing. Command-line arguments identify the input, reference, output, and log directories, allowing Airflow and local runs to invoke the same implementation rather than duplicating transformation logic.

The job writes partitioned processed activity plus hourly, daily, geospatial, and hotspot analytics. It logs input rows, rejected rows, handled nulls, output rows, and execution status in machine-readable key-value form. Exceptions propagate as failed executions while Spark shutdown is managed centrally. This module becomes the processing unit orchestrated in Phase 3.

7 Phase 3 Data Engineering

Phase 3 converted the transformation scripts into a repeatable production-style data pipeline. The work introduced named data zones, deterministic routing, task orchestration, dimensional warehouse loading, operational status, freshness checks, and failure-aware execution. The accumulated history created by this phase supplies the API and machine-learning layers.

7.1 DE1 Telecom Data Architecture

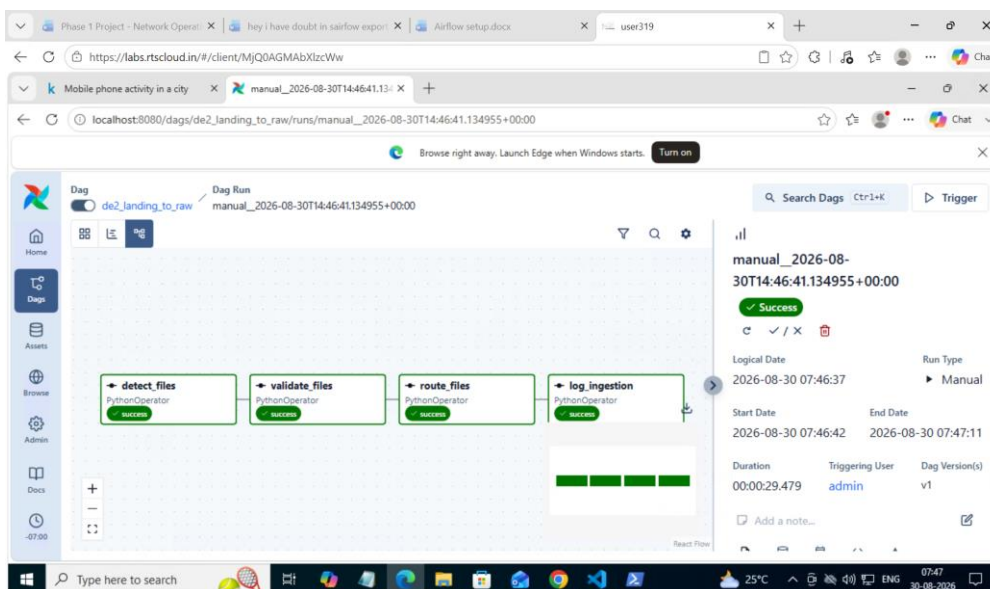
A layered architecture was defined around landing, raw, rejected, reference, processed, analytics, warehouse, service, and presentation concerns. The landing zone represents unprocessed arrivals. Accepted files are copied unchanged to raw storage, while invalid files are copied to rejected storage with a reason. Static geographic reference data is isolated from daily activity. Spark writes clean and aggregated Parquet products to processed and analytics locations, and the warehouse stores reusable dimensions and facts for application queries.

The design separates transient arrival handling from durable evidence and curated products. It also establishes clear ownership: Airflow coordinates tasks, Spark performs distributed transformations, MySQL supports relational retrieval, FastAPI defines service contracts, React owns the user experience, and ML modules operate on curated history. This prevents raw-file access from leaking into downstream applications.

7.2 DE2 Landing to Raw Ingestion

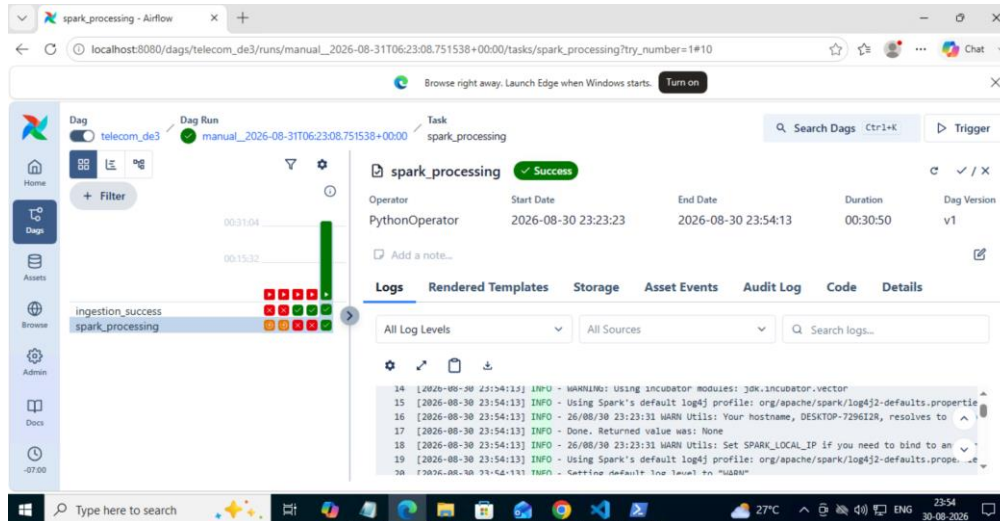
de2_ingestion.py implements the landing-to-raw control. It creates the landing, raw, rejected, and log directories; detects the expected Milan activity file; validates required columns; checks timestamp parseability and CellID validity; and records the row count and reason associated with each decision. Accepted and rejected files are copied with metadata preserved rather than transformed in place.

Idempotency is handled by detecting whether the target file already exists. Repeated delivery is reported as `ALREADY_PROCESSED` and does not overwrite prior evidence. Structured log entries capture filename, status, row count, and reason. A DE2 Airflow DAG exposes detection, validation, routing, and logging as ordered Python tasks, making the arrival process observable through the scheduler.



7.3 DE3 Spark Job Orchestration

The DE3 Airflow workflow verifies that accepted files exist in raw storage before invoking the reusable Spark pipeline. The Spark job is executed through a subprocess with explicit input, reference, output, and log arguments. Standard output, standard error, and the exit code are captured, and any nonzero result is raised as a task failure. The dependency `ingestion_success` followed by `spark_processing` prevents transformation from starting without a valid upstream arrival.



7.4 DE4 Batch and Streaming Decision

Batch processing was selected as the appropriate operating model for the supplied project data. Files arrive as complete daily extracts, the analytical grain is hourly, and there is no event broker or sub-hour operational latency requirement. A scheduled or arrival-triggered batch therefore provides simpler recovery, reproducibility, and cost control than introducing a continuous streaming stack.

The decision does not prevent future streaming. The canonical schema, grid-hour contract, data-quality rules, and service layer could be retained if a message source were introduced. The main changes would be watermarking, state management, late-data handling, checkpointing, and incremental feature updates. For the implemented scope, Airflow-managed micro-batches or daily batches provide the clearest and most supportable design.

7.5 DE5 Storage Strategy and Data Zones

Storage formats were matched to purpose. CSV is retained for external delivery and compact dashboard extracts; Parquet is used for processed and analytical datasets because it preserves schema, compresses numeric columns, and supports selective reads; GeoJSON remains the authoritative geographic reference and static browser asset; JSON is used for machine-readable pipeline status; and MySQL provides indexed relational access for APIs and model features.

Partitioning is applied where it improves common access patterns. The clean activity dataset is partitioned by date, while compact summaries are stored without excessive fragmentation. Geometry is held in the reference and specialized geospatial products instead of being repeated in every hourly analytical record. This reduces storage and avoids moving large polygon text through queries that only need numeric activity.

7.6 DE6 Warehouse Modelling

A dimensional MySQL warehouse was created with `dim_time`, `dim_grid`, and `fact_network_activity`. `dim_time` stores a stable time key, timestamp, date, hour, and day of week. `dim_grid` stores a surrogate key, business `grid_id`, centroid latitude and longitude, and a reference to the source geometry. `fact_network_activity` stores one grid-hour observation with the individual activity measures and derived totals.

Foreign keys link the fact table to both dimensions, and a unique constraint on `time_key` and `grid_key` enforces the operational grain. Indexes support time filtering, grid filtering, combined grid-time access, and dimension lookups. Data is inserted in large batches, while `ON DUPLICATE KEY UPDATE` makes reruns idempotent. The grid dimension is populated from `properties.cellId` and computed polygon centroids; the time and fact tables are loaded from the Spark hourly summary. Warehouse validation checks dimension and fact counts, referential coverage, duplicate grain, null keys, and aggregate consistency.

7.7 DE7 End to End Airflow Orchestration

The end-to-end DAG coordinates ingest, validate, `spark_process`, `load_warehouse`, `quality_check`, and notify tasks. Each task records a timestamped status and relevant metrics in a run-specific JSON state file. Existing scripts are invoked rather than reimplemented, so the same code paths are used for direct and scheduled execution.

The quality-check task confirms that the hourly analytics output exists and contains rows, calculates AS OF from the maximum actual data timestamp, checks required columns and null keys, reads Spark metrics from the processing log, assesses file-publication freshness, and compares the published output with warehouse state. A consolidated `pipeline_status.json` contains the run identifier, task states, rows in, rows rejected, nulls handled, rows published, warehouse rows, freshness indicator, health flag, and reasons. Atomic temporary-file replacement prevents consumers from reading a partially written status record.

7.8 DE8 Pipeline Reliability

Reliability is implemented through early validation, explicit exceptions, idempotent routing, unique warehouse keys, task dependencies, structured logs, and machine-readable run state. A failure in detection, validation, Spark processing, warehouse loading, or quality checks is recorded against the responsible task and propagated to Airflow. The final notification task uses an all-done trigger so a terminal status can still be written when an upstream task fails.

The pipeline distinguishes freshness from data time. AS OF represents the latest observation in the dataset, while freshness represents the age of the published analytical files. This prevents stale processing from appearing current simply because the historic dataset has a valid maximum timestamp. Safe reruns, count reconciliation, duplicate checks, and status reasons provide the evidence needed to investigate failures without silently accepting partial results.

8 Phase 4 FastAPI Service Layer

Phase 4 exposed the warehouse, features, alerts, model, geography, and pipeline state through a versioned FastAPI application. Routers separate endpoint definitions from service logic, Pydantic models enforce response structure, database connections are opened for a request and closed deterministically, and CORS permits the React development application to consume the service.

8.1 API1 Network Summary Endpoint

GET /network/summary returns total_activity, active_grids, peak_hour, top_grid, and as_of. If a cutoff is not supplied, the service resolves the maximum warehouse timestamp. It then calculates the latest valid time key at or before that cutoff and performs SQL aggregations over the fact table. Active grids are counted only where activity is positive, peak hour is determined across the selected history, and the highest-activity grid is resolved through dim_grid.

The endpoint wraps the query in a stable NetworkSummary response model. An unavailable analytical source is returned as a server error with a clear message, while an empty cutoff range produces a neutral summary. The use of the warehouse AS OF value ensures the API does not substitute the server clock for the data's reporting time.

8.2 API2 Grid Activity Drill Down

GET /network/grid/{grid_id} validates that the identifier is numeric, falls within the 1 to 10000 range, and exists in dim_grid. Optional date, hour, and as_of parameters narrow the query. The service returns the selected grid, effective reporting time, start and end times, and a chronological collection of SMS, call, internet, and total activity points.

Unknown grids produce a 404 response and invalid hours are rejected. The frontend can therefore distinguish a legitimate empty interval from an invalid geographic cell. All plotted series are retrieved from the warehouse rather than recomputed in React.

8.3 API3 Hotspots and Alerts

GET /network/hotspots ranks activity at the effective AS OF timestamp and supports a bounded limit. Each result includes the component measures, total activity, status, severity, reason, and optional risk fields. GET /network/alerts retrieves stored NP3 alerts up to the cutoff, joins their activity context, supports minimum-severity filtering, and exposes the alert type, current value, baseline, severity, and reason.

Both services left-join network_anomaly_scores so ML3 risk_score, risk_level, and model_version can be added without breaking the original contracts. This additive design allows the React map and tables to combine deterministic operational signals with model-based prioritization while retaining the source reason for each item.

8.4 API4 Grid Feature Endpoint

GET /network/grid/{grid_id}/features retrieves the most recent engineered feature record at or before an optional cutoff. The response includes avg_activity, median_activity, median_avg_ratio, activity_growth, active_hours, peak_ratio, variability, internet_share, hour_of_day, and day_of_week. The API also adds simple data-quality and feature-freshness descriptors without recalculating the features.

The strict correspondence between the stored ML2 field names, Pydantic model, and prediction-service feature order is essential. Keeping feature calculation outside the API prevents online and batch inference from diverging.

8.5 API5 Prediction Endpoint

POST /network/predict-risk accepts a grid_id and feature_timestamp. The compact request contract allows the service to fetch the authoritative stored features instead of trusting arbitrary client-supplied model values. The response contains ML3 risk score and level, ML4 anomaly score and direction when available, NP3 alert indicators, a comparison category, model version, and a concise explanatory note.

The route preserves the contract used by the React predictive page while the implementation behind it is supplied by ML5. Missing features are returned as a 404 condition, invalid numerical feature values are rejected, and inference metadata remains separate from any future natural-language explanation capability.

8.6 API6 Operational Support Endpoints

GET /pipeline/status reads the status record written by the end-to-end Airflow DAG and compares its AS OF value with the current warehouse maximum. Task statuses are evaluated and any mismatch or failed task is added to the reasons list before the final healthy flag is returned. This makes the pipeline's own operational evidence available to applications without reconstructing health independently.

GET /network/grid/{grid_id}/location returns the stored centroid and geometry reference for one cell. GET /network/grid/{grid_id}/neighbours ranks other cells by centroid distance and returns a configurable number of nearby grids. These endpoints support geographic investigation while avoiding transmission of the full polygon geometry in every location request.

Network Operations Analytics API 1.0.0 OAS 3.1

/openapi.json

FastAPI service for telecom network analytics and NOC KPIs.

Network

GET	/network/summary	Network Summary	^
GET	/network/grid/{grid_id}	Grid Activity	^

Network Alerts

GET	/network/hotspots	Hotspots	^
GET	/network/alerts	Alerts	^

Grid Features

GET	/network/grid/{grid_id}/features	Grid Features	^
-----	----------------------------------	---------------	---

Risk Prediction

POST	/network/predict-risk	Predict Network Risk	^
------	-----------------------	----------------------	---

Operational Support

GET	/pipeline/status	Pipeline Status	^
GET	/network/grid/{grid_id}/location	Grid Location	^
GET	/network/grid/{grid_id}/neighbours	Grid Neighbours	^

default

GET	/	Root	^
-----	---	------	---

9 Phase 5 React NOC Dashboard

Phase 5 implemented a Vite-based React application that consumes the FastAPI contracts and presents the platform as a lightweight Network Operations Center. BrowserRouter provides page navigation, a shared service module centralizes API calls, and each page implements its own loading, success, empty, and error states. The interface uses Recharts for time-series exploration and React Leaflet for geographic visualization.

9.1 RE1 Dashboard Setup

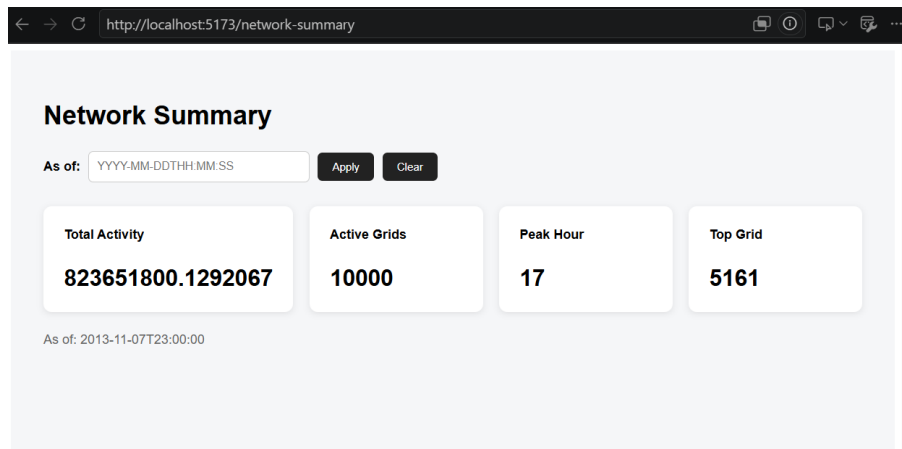
The React application is configured through `VITE_API_BASE_URL`, which keeps the service location outside component code. FastAPI CORS middleware enables browser requests during development. Routes were created for the home page, network summary, grid explorer, hotspots and alerts, and predictive risk. The home screen acts as an operational navigation hub with a clear description of each area.

The first end-to-end connection is the network summary request. React state tracks data, loading, and failure conditions, and the page is rendered conditionally according to request status. This pattern is reused across the other pages and provides consistent feedback when the API is unavailable.

9.2 RE2 Network Overview

The Network Summary page calls `/network/summary` on initial load and allows an optional reporting cutoff to be submitted or cleared. Four KPI cards present the composite total activity, active grids, peak hour, and top grid. The page displays the API's `as_of` timestamp so the user sees the time represented by the dataset rather than the browser clock.

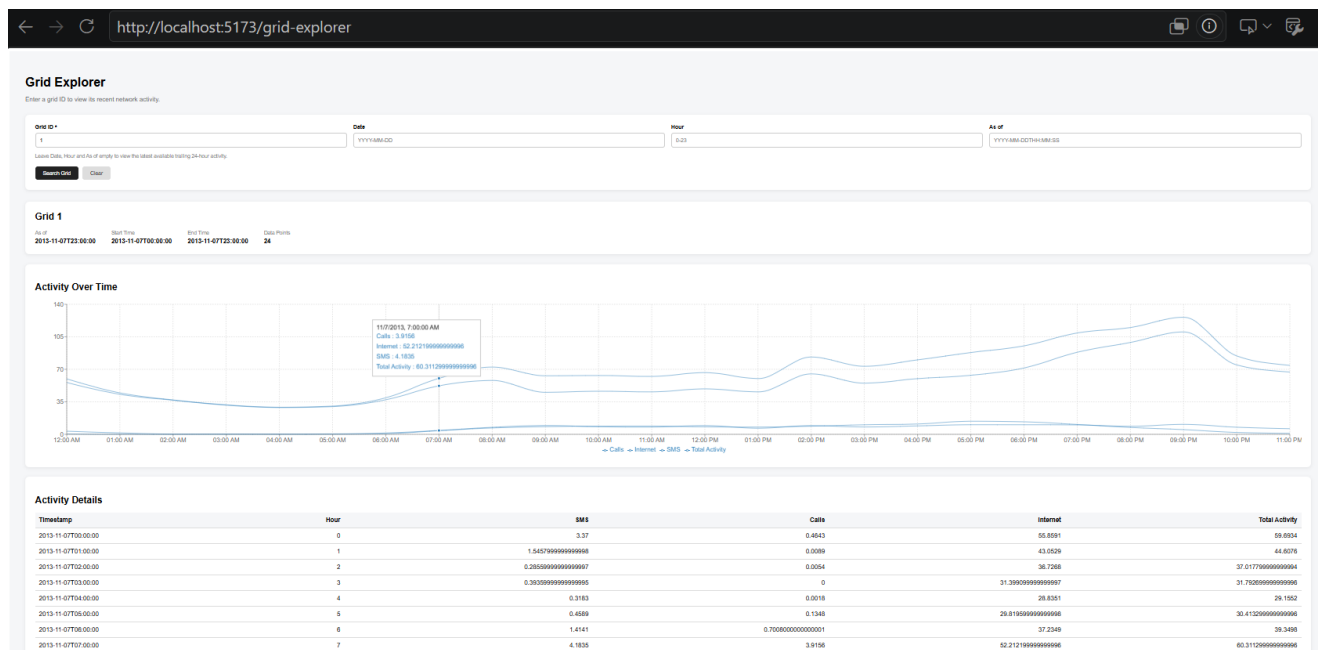
The page contains explicit loading and error states and refreshes its cards only after a successful response. The labels preserve the activity-indicator semantics and do not describe the values as message counts, call counts, or network capacity.



9.3 RE3 Grid Explorer

The Grid Explorer provides controlled inputs for grid ID, date, hour, and AS OF. It validates the required grid value and the zero-to-23 hour range before building the query string. A grid selected on the Milan map can be passed through the `grid_id` URL parameter and preloaded into the explorer.

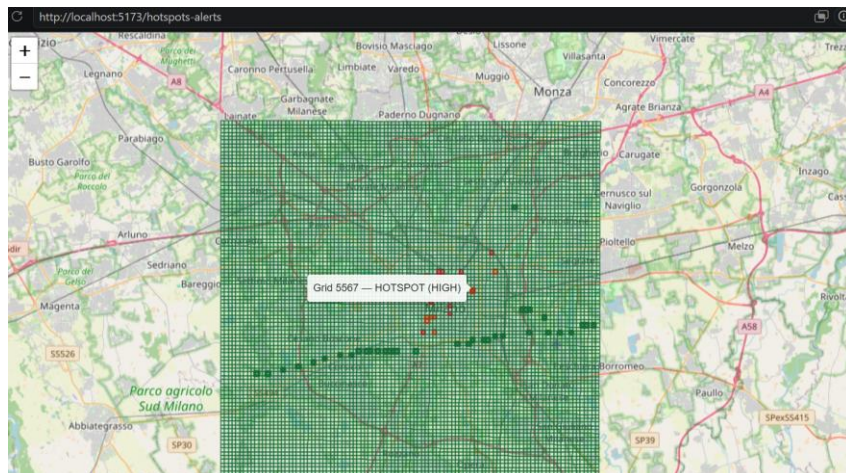
Returned observations are displayed in a Recharts line chart and a detailed activity table. Separate series expose SMS, calls, internet activity, and the composite total over time. The page also presents the grid, effective time range, and reporting timestamp, while unknown grids and server errors are shown without breaking the dashboard.



9.4 RE4 Hotspots Alerts and Milan Map

The Hotspots and Alerts page loads the static Milan GeoJSON once into React state and retrieves ranked hotspots and alerts from the API. Independent limits, severity, and AS OF controls allow the operational list to be narrowed. The tables display grid, timestamp, activity or alert type, severity, and available ML3 risk information.

API results are normalized by grid_id and joined to GeoJSON features through properties.cellId. Leaflet renders the actual cell polygons rather than approximate points. Polygon styling distinguishes cells associated with hotspots or alerts, hover interactions highlight the feature, and a selected cell can open the Grid Explorer. Keeping the 10,000-cell geometry as a static browser resource avoids repeatedly sending large polygon data through database-backed endpoints.



9.5 RE5 Predictive Risk View

The Predictive Risk page accepts a grid ID and feature timestamp and submits them to POST /network/predict-risk. It validates both fields locally, shows progress during inference, and reports API errors clearly. The ML3 probability is formatted as a percentage and paired with a HIGH or LOW risk label and model version.

Additional fields present the ML4 anomaly score and direction, NP3 alert presence and count, alert types, and the three-way comparison category. Model output is visually separated from the later Explain with AI placeholder. This separation reinforces that the numeric model and rules are evidence sources, while a future language model would only explain curated results rather than create them.

Predictive Risk

Request a model-based risk prediction for a selected network grid and timestamp.

Prediction Request

Grid ID

1

Feature Timestamp

2013-11-04T14:00:00

Predict Risk

Clear

Model Output

ML3 Risk Score

62.83%

HIGH

Grid ID	Feature Timestamp	Model Version
1	2013-11-04T14:00:00	ML3_XGBOOST_V1

Additional Model Results

Anomaly Score	Anomaly Direction	NP3 Alert
23.218615374903877	NORMAL	0
NP3 Alert Count	NP3 Alert Types	Comparison
0	-	NONE

10 Phase 6 Machine Learning

Phase 6 added lightweight, operationally interpretable intelligence to the curated activity history. Features were computed with past-only Spark windows, models were evaluated with a chronological split, a historical same-grid and same-hour baseline supplied independent anomaly evidence, and the selected model was operationalized through FastAPI. Positive predictions are treated as prompts to investigate, not as confirmation of congestion or failure.

10.1 ML1 Operational Machine Learning Problem

The primary supervised task was framed as next-hour high-activity risk for a grid. The prediction unit is one grid and one feature timestamp t . Every feature is calculated from the trailing history ending at t , while the target describes activity at t plus one hour. This boundary turns the task into a forecast rather than a restatement of the current value and provides the central protection against temporal leakage.

The implemented target is a transparent proxy: HIGH_ACTIVITY equals one when next-hour activity is at least 1.20 times the trailing median activity. The threshold describes unusual elevation relative to recent behavior, not congestion. A positive result means that the grid-hour deserves operational investigation. Historical anomaly detection was retained as a second mechanism because an observation can be abnormal for its usual hour even when the forecast classifier does not place it in the high-activity class.

10.2 ML2 Feature Engineering

ml2_features_pyspark.py reads total and internet activity from the MySQL fact and dimension tables through JDBC. Invalid keys and negative values are removed, and the numerical measures are normalized before window calculations. For each grid, a recent window contains the latest 24 rows through t , while a separate 24-row baseline window covers the period immediately before the recent window. Rows are retained only after a complete 48-hour history is available.

The resulting feature vector captures level, shape, trend, composition, and calendar context. avg_activity and median_activity represent the recent level. median_avg_ratio describes distribution shape. activity_growth compares the recent mean with the preceding baseline mean. active_hours counts positive observations. peak_ratio compares the recent maximum with the mean. variability uses population standard deviation. internet_share measures the internet component of recent total activity. hour_of_day and day_of_week represent recurring temporal patterns.

feature_timestamp is set to t , the final hour visible to the feature calculation. All Spark windows end at the current row and never include future rows. The features are persisted in network_feature_new_final, and a dedicated leakage test module is included so changes that allow post- t information into earlier features can be detected.

Feature	Operational interpretation
avg_activity	Mean composite activity across the trailing 24 hours
median_activity	Robust recent activity baseline
median_avg_ratio	Relationship between the median and mean, indicating skew or peaks
activity_growth	Relative change from the preceding 24-hour baseline window

Feature	Operational interpretation
active_hours	Number of positive-activity hours in the recent window
peak_ratio	Recent maximum divided by recent average
variability	Population standard deviation of recent activity
internet_share	Recent internet activity divided by recent total activity
hour_of_day and day_of_week	Recurring temporal context available at prediction time

10.3 ML3 Risk Classifier

The engineered feature table was joined to fact_network_activity by grid_id and a target timestamp exactly one hour after feature_timestamp. Records without a matching future observation were removed. The resulting data was ordered by feature time, unique timestamps were divided chronologically at the 80 percent point, and earlier observations were used for training while later observations were reserved for testing. This preserves the direction of time and avoids the optimistic leakage that a random split can produce when neighboring observations are strongly related.

The prepared dataset contained 1,209,994 feature records before the final target alignment. The training partition contained 959,991 labeled records, including 303,270 positives for a 31.59 percent positive base rate. The test partition contained 239,998 records, including 67,597 positives for a 28.17 percent base rate. Infinite values were replaced with missing values, and medians calculated only from the training partition were used to fill both training and test data.

Logistic Regression, Decision Tree, and XGBoost classifiers were compared. Logistic Regression was scaled and class-balanced. The Decision Tree was depth- and leaf-constrained for stability and interpretability. XGBoost used 500 estimators, a maximum depth of six, a learning rate of 0.05, subsampling, and column sampling. A probability threshold of 0.35 was applied to the selected XGBoost model to balance precision and recall for the operational task.

Model	Accuracy	Precision	Recall	F1	Test base rate
Logistic Regression	73.02%	51.35%	80.27%	62.63%	28.17%
Decision Tree	80.26%	83.54%	37.24%	70.26%	28.17%
XGBoost	84.18%	69.40%	78.40%	73.63%	28.17%

XGBoost achieved the strongest F1 score and was saved as the operational model together with the ordered feature list, target name, decision threshold, and training and test time ranges. Feature importance showed `hour_of_day` as the dominant feature at 0.471, followed by `median_avg_ratio` at 0.155 and `day_of_week` at 0.104. Peak ratio, growth, internet share, variability, median activity, and average activity provided additional signal, while `active_hours` contributed no split importance in the selected model. The evaluation report and feature-importance table were also exported for review.

The metrics were interpreted together rather than relying on accuracy alone. Compared with the 28.17 percent test base rate, the selected model provides substantial discrimination, but its 69.40 percent precision still implies false operational attention signals. Its 78.40 percent recall shows that most positive proxy events are detected, while some remain missed. These results support prioritization and investigation rather than automated fault declaration.

10.4 ML4 Historical Anomaly Baseline

ML4 provides a second intelligence mechanism with different assumptions. For each grid and hour of day, the baseline at time *t* is the expanding median of observations from previous days at the same hour. The current observation is shifted out of the baseline calculation, so it cannot influence its own reference. The implementation records the amount of preceding same-hour history, defines two days as the intended minimum-history setting, and labels rows without a usable baseline as having insufficient historical data.

The lowest 15 percent of grid-days by daily activity are removed to reduce unstable percentage ratios. Percentage deviation is calculated as current activity minus baseline divided by baseline, and its absolute value becomes the anomaly score. A deviation of at least positive 30 percent is classified as HIGH, a deviation of at most negative 30 percent is classified as LOW, and values inside the band are NORMAL. Every classification receives a reason containing the current value, historical baseline, and magnitude of deviation.

ML3 predictions are shifted forward one hour so their predicted interval aligns with the observed ML4 timestamp. NP3 alerts are grouped by grid and timestamp and joined to the same record. The comparison layer categorizes ALL_AGREE, pairwise agreement, ML3_ONLY, ML4_ONLY, NP3_ONLY, or NONE and generates an explanation for each case. This treats disagreement as useful evidence: ML3 forecasts elevated future activity, ML4 measures deviation from a time-specific historical norm, and NP3 applies simple within-day rules.

Combined results are written in batches to `network_anomaly_scores` with current and baseline activity, anomaly score, direction, reason, ML3 risk probability and level, NP3 alert details, comparison category, comparison reason, and model metadata. The table becomes the shared operational product used by prediction, hotspot, and alert services.

10.5 ML5 Model Operationalization through FastAPI

The prediction service loads `ml3_best_model.pkl` during application startup and fails clearly if the artifact is missing. The package supplies the trained estimator, feature order, model name, and decision threshold. For an incoming grid and feature timestamp, the service retrieves the authoritative stored feature row, creates the numerical vector in the exact training order, verifies that all values are finite, and calculates the positive-class probability with `predict_proba`.

The risk level is assigned by comparing the probability with the stored threshold. The service also queries the anomaly table for the corresponding evidence and returns ML4, NP3, and comparison fields when available. The model version is included in every response. Because the request and response contracts remain stable, the React predictive page consumes real inference without needing a separate client implementation for the trained model.

10.6 ML6 Batch Grid Scoring

Batch inference is implemented by loading the complete stored feature history, generating ML3 probabilities for all eligible grid-time rows, shifting each prediction to its future observation timestamp, and joining the results to ML4 anomalies and NP3 alerts. The combined score rows are inserted into `network_anomaly_scores` in 5,000-row batches after the destination schema is created and refreshed. Progress and summary counts are printed so large scoring runs remain observable.

The batch product is integrated into the application through additive API fields. The hotspot service joins activity to the score table at the same grid and timestamp, and the alert service performs the same join for rule-based events. Risk score, risk level, and model version therefore appear in the operational tables and Milan map without changing the original activity and alert semantics. Direct API inference remains available for a selected grid and time, while the batch table supports ranking and cross-mechanism comparison across the full history.

The completed Phase 6 implementation closes the analytical loop: curated warehouse history produces leakage-controlled features; those features produce next-hour risk probabilities; observed activity is compared with historical same-hour behavior; rule, model, and anomaly evidence are aligned; and the combined results are served through FastAPI and displayed in React. The final system remains intentionally decision-support oriented, with high scores and anomalies presented as investigation signals rather than confirmed congestion or faults.